

RED TEAM LEARNING PATH

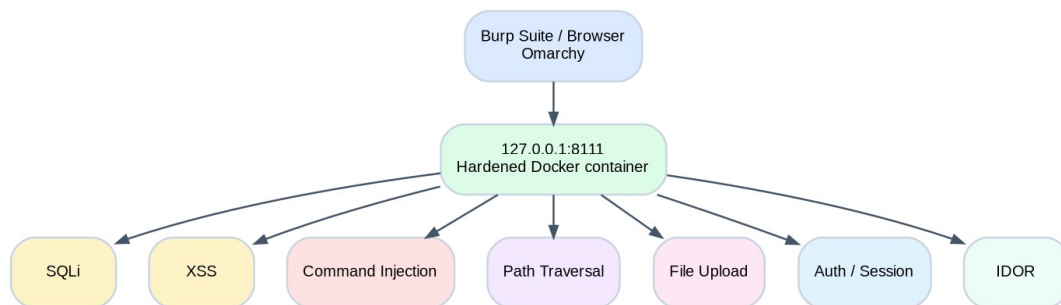
MODULE 11

Web Vulnerabilities I

SQL Injection • XSS • Command Injection • Path Traversal • File Upload • Authentication • Access Control / IDOR

AUTHORIZED LABS ONLY

Semua payload dan langkah eksploitasi pada modul ini ditujukan untuk local Docker lab milik lo sendiri atau lab PortSwigger yang memang sengaja vulnerable. Jangan menguji aplikasi publik atau pihak lain tanpa authorization tertulis.



Peta local training lab Module 11

Prerequisites: Module 00-10 • Burp Suite fundamentals • HTTP basics • Docker

Field	Value
Prerequisite	Module 00–10 — Burp Suite fundamentals, HTTP basics, Docker
Core tools	Burp Suite, curl, Docker
Local target	http://127.0.0.1:8111/
Outcome	Mengenali pola akar masalah tujuh kelas vulnerability web umum, memvalidasinya manual, menjelaskan impact tanpa melebih-lebihkan, dan memberi remediation yang menasar root cause

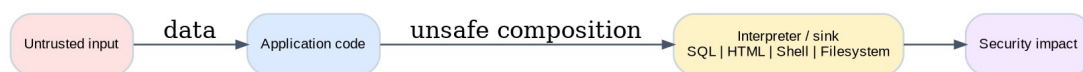
1. Learning Objectives

- [] Membedakan **source**, **transformation**, **sink**, dan **security decision** dalam request flow.
- [] Menemukan SQL injection sederhana dan menjelaskan kenapa parameterized query mencegahnya.
- [] Membedakan reflected XSS, stored XSS, dan DOM-based XSS secara konseptual.
- [] Mengonfirmasi OS command injection pada container lab yang terisolasi.
- [] Mengidentifikasi path traversal dan menjelaskan canonicalization/path allow-listing.
- [] Menguji file upload berdasarkan extension, MIME, storage, dan serving behavior.
- [] Membedakan authentication flaw dari authorization flaw.
- [] Menguji IDOR dengan membandingkan dua object identity di Burp Repeater.
- [] Menulis finding note berisi evidence, impact, dan remediation.

Pola pikir utama.

Jangan mulai dari payload. Mulai dari pertanyaan: input mana yang gue kontrol, interpreter/keputusan apa yang menerima input itu, dan perubahan apa yang membuktikan boundary keamanan benar-benar rusak?

2. Model Mental: Source → Sink → Security Impact



Trust boundary yang sering rusak pada vulnerability web

Banyak vulnerability web terlihat berbeda tapi berbagi pola yang sama: data tidak dipercaya mencapai interpreter, output context, filesystem path, atau authorization decision dengan pemisahan *data-vs-code* / *identity-vs-permission* yang salah.

Kelas	Source umum	Sink/keputusan	Pertanyaan tester
SQLi	Query/body/header	SQL interpreter	Apakah input mengubah struktur query?
XSS	Query/form/API	HTML/JS/DOM sink	Apakah data menjadi executable browser content?
Command Injection	Parameter/form	Shell/process invocation	Apakah input mengubah command/argument?
Path Traversal	Filename/path	Filesystem resolution	Bisakah user keluar dari directory yang diizinkan?
Auth flaw	Credential/session data	Identity decision	Bisakah identity dipalsukan/dilewati?

IDOR	Object identifier	Authorization decision	Apakah object access diverifikasi server-side?
------	-------------------	------------------------	--

3. Lab Setup & Hardening

```
cd modules/11-web-vulnerabilities-i/labs
./workspace-init.sh
./vuln-web-lab.sh build
./vuln-web-lab.sh create
./vuln-web-lab.sh status
```

Baseline:

```
curl -i http://127.0.0.1:8111/
# atau buka URL yang sama di Burp built-in browser
```

Launcher menjalankan aplikasi yang sengaja vulnerable dengan hardening berikut, sehingga eksploitasi tetap nyata **di dalam** container tapi tidak punya host mount atau privileged access:

```
--cap-drop ALL
--security-opt no-new-privileges:true
--read-only
--tmpfs /tmp:rw,noexec,nosuid,size=64m
-p 127.0.0.1:8111:8111 (tidak dipublish ke LAN)
```

Security Note.

Jangan ubah binding launcher dari 127.0.0.1 menjadi 0.0.0.0. Aplikasi ini sengaja vulnerable dan tidak boleh diekspos ke LAN atau internet.

4. SQL Injection (SQLi)

SQL injection terjadi ketika aplikasi membangun query SQL dengan menggabungkan input user ke kode query — input yang seharusnya data berubah menjadi syntax SQL dan memodifikasi maksud query.

Pola code smell di app.py (endpoint /products):

```
category = request.args["category"]
query = f"SELECT id,name,category FROM products WHERE category = '{category}' AND hidden = 0"
```

Root cause.

Masalah utamanya **bukan karakter petik**. Masalahnya adalah code dan data digabung menjadi satu string yang kemudian diparse database sebagai SQL. Prepared/parameterized query memisahkan struktur query dari nilai data.

Hands-on 4.1 — Baseline → SQLi

- Buka /products?category=Gifts di browser/Burp, kirim ke Repeater.
- Catat baseline status, response length, dan product yang muncul (dua produk visible, satu hidden=1 tersembunyi).
- Ubah **hanya** parameter category dengan payload training.
- Bandingkan response: apakah hidden product (Internal Prototype) ikut muncul?
- Kembalikan parameter ke baseline dan pastikan behavior normal kembali.

```
Payload local lab: ' OR 1=1--
Versi URL-encoded: http://127.0.0.1:8111/products?category=%27%20OR%201%3D1--
```

Evidence	Apa yang dicatat
Baseline	request + daftar product normal
Modified	parameter tunggal yang berubah
Delta	hidden row/result count bertambah
Root cause	query string concatenation

Fix

prepared statement/parameter binding + least privilege

Remediation.

Gunakan prepared statements/parameterized queries. Allow-list validation bisa jadi defense-in-depth, tapi bukan pengganti parameterization. Database account juga harus least privilege.

5. Cross-Site Scripting (XSS)

XSS muncul ketika untrusted data ditulis ke HTML/JavaScript/DOM context tanpa output encoding/sanitization yang sesuai. **Context matters** — encoding untuk HTML body tidak otomatis aman untuk JavaScript, URL, atau HTML attribute context.

Jenis	Lokasi input	Lokasi eksekusi
Reflected	request saat ini	response langsung
Stored	disimpan server	response berikutnya / user lain
DOM-based	client-side source	browser DOM sink

Endpoint lab /search me-reflect q mentah-mentah ke HTML body (app.py):

```
return f'<!doctype html><html><body>...<p>Results for: {q}</p>...'
```

Hands-on 5.1 — Reflected XSS

Baseline:
http://127.0.0.1:8111/search?q=hello

Benign local proof-of-execution:
<svg onload=alert(document.domain)>

Kirim parameter melalui Repeater atau browser. Tujuannya hanya membuktikan browser mengeksekusi markup/JavaScript yang berasal dari input user. **Jangan** memasukkan cookie exfiltration atau network callback.

Remediation.

Gunakan context-aware output encoding; sanitization hanya ketika aplikasi memang harus menerima HTML. Modern framework auto-escaping membantu, tapi unsafe escape hatches tetap berbahaya. CSP adalah defense-in-depth, bukan root-cause fix.

6. OS Command Injection

OS command injection terjadi saat aplikasi membangun command shell dari input user. Dampaknya bisa sangat tinggi karena attacker-controlled input dapat menjadi syntax shell. Di lab ini, eksekusi dibatasi ke container disposable dengan privilege minimal.

Endpoint lab /system (app.py):

```
cmd = 'printf "checking %s\\n" ' + host
subprocess.check_output(cmd, shell=True, ...)
```

Hands-on 6.1 — Confirm Shell Interpretation

```
GET /system?host=127.0.0.1 HTTP/1.1
Host: 127.0.0.1:8111
```

Payload training: 127.0.0.1;id

Kirim via Repeater dan cari output uid/gid container user — bukti bahwa shell separator (;) mengubah satu nilai menjadi command tambahan.

Common mistake.

Mencoba "sanitize" semua shell metacharacter biasanya rapuh. Defense utama adalah **menghindari shell sama sekali** dan menggunakan library/API yang sesuai (subprocess tanpa shell=True, argument list eksplisit); kalau proses eksternal wajib, pass executable dan argument secara terstruktur serta validasi allow-list.

Blue Team Perspective.

Perhatikan process creation dari web application user, command line yang tidak biasa, shell child process dari application server, dan unexpected outbound behavior.

7. Path Traversal

Path traversal muncul ketika user mengontrol sebagian filesystem path dan aplikasi tidak membatasi hasil path resolution ke directory yang memang diizinkan. String filtering `../` saja sering gagal karena encoding/canonicalization dan platform path semantics.

Endpoint lab `/download (app.py)`:

```
path = Path('/app/files/' + name) # name berasal langsung dari ?file=
```

Hands-on 7.1 — Leave the Intended Directory

```
Baseline: http://127.0.0.1:8111/download?file=welcome.txt
Local container proof: ../../etc/hostname
```

Tujuan exercise adalah membaca file harmless dari filesystem container di luar `/app/files`. **Jangan mengejar secrets** — bukti cukup menunjukkan boundary directory bisa dilewati.

Remediation.

Jangan menerima arbitrary path. Map user-visible identifier ke server-side filename, resolve/canonicalize path, lalu verifikasi hasilnya tetap berada di allowed base directory sebelum membuka file.

8. File Upload Vulnerabilities

File upload security bukan sekadar extension. Tester perlu melihat siapa yang boleh upload, filename handling, extension/type/content validation, storage location, permission, size, processing pipeline, dan bagaimana file kemudian disajikan kembali.

Endpoint lab `/upload (app.py)` menyimpan file apa adanya ke directory yang kemudian di-serve same-origin lewat `/uploads/<name>`:

Kontrol	Pertanyaan
Extension	Allow-list atau blacklist? case handling?
MIME	Apakah server hanya percaya Content-Type dari user?
Filename	Direname? collision? traversal chars?
Storage	Di web root atau storage terpisah?
Serving	Content-Disposition? MIME sniffing? same-origin?
Processing	Image parser/archive extractor/document parser?

Hands-on 8.1 — Harmless HTML Upload

```
cat > /tmp/module11-proof.html <<'EOF'
<!doctype html><title>upload proof</title>
<h1>Module 11 upload proof</h1>
<script>document.body.dataset.executed="yes"</script>
EOF
```

Upload lewat `/upload`, lalu buka link hasil upload. Observasi bahwa aplikasi menerima HTML dan menyajikannya kembali dari same origin — ini menunjukkan weak validation/serving policy **tanpa perlu web shell**.

PortSwigger extension.

Untuk belajar server-side executable upload, gunakan hanya Web Security Academy *file-upload* labs yang memang sengaja vulnerable. Jangan mengupload web shell ke sistem lain.

Remediation.

Allow-list extension yang benar-benar dibutuhkan; validasi type/content secara independen; rename file; limit size; simpan di storage terpisah/non-executable; serve dengan safe Content-Type/Content-Disposition; scan/process secara terisolasi.

9. Authentication & Session Integrity

Authentication membuktikan identity. Session membawa identity itu antar-request. Kalau aplikasi mempercayai client-controlled identity state **tanpa integrity protection**, attacker dapat memalsukan siapa dirinya bahkan tanpa menebak password.

Endpoint lab /login → /profile (app.py) memercayai cookie user= mentah tanpa signature:

```
r.set_cookie('user', 'student', httponly=True, samesite='Lax') # tidak signed
...
user = request.cookies.get('user', 'guest')
if user == 'admin': return '<h2>Admin profile</h2>...'
```

Hands-on 9.1 — Unsigned Identity Cookie

- Login ke /login menggunakan student / student.
- Capture request ke /profile dan kirim ke Repeater.
- Baseline Cookie: user=student menghasilkan student profile.
- Ubah **hanya** cookie menjadi user=admin.
- Kalau admin profile muncul, dokumentasikan bahwa server mempercayai identity dari unsigned client cookie.

```
Single-variable test:
Cookie: user=student
# ubah menjadi
Cookie: user=admin
```

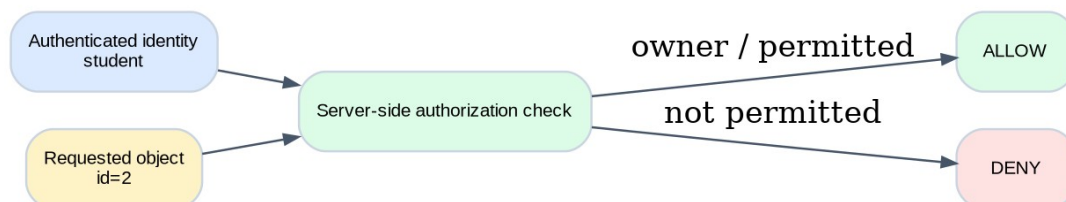
Authentication ≠ Authorization.

AuthN menjawab "siapa lo?", AuthZ menjawab "lo boleh melakukan apa terhadap resource ini?". Keduanya harus divalidasi server-side.

Remediation.

Gunakan framework session mechanism yang cryptographically protected/server-side; rotate session setelah authentication; cookie Secure/HttpOnly/SameSite sesuai konteks; jangan jadikan user-controlled field sebagai proof of identity.

10. Access Control & IDOR



Authorization harus mengikat identity + action + object, bukan hanya "user sudah login"

IDOR adalah salah satu bentuk access-control failure. Object identifier boleh saja predictable — yang wajib adalah server melakukan authorization untuk setiap object/action berdasarkan authenticated identity dan policy.

Endpoint lab /api/profile (app.py) mengambil id dari query string tanpa cek siapa yang bertanya:

```
user_id = request.args.get('id', '1')
row = con.execute('SELECT id,username,role,bio FROM users WHERE id=?',(user_id,)).fetchone()
# Intentionally missing object-level authorization check.
```

Hands-on 10.1 — Horizontal Object Access

```
# Baseline object
GET /api/profile?id=1 HTTP/1.1
Host: 127.0.0.1:8111

# Change one object identifier
GET /api/profile?id=2 HTTP/1.1
Host: 127.0.0.1:8111
```

Bandingkan response. Kalau request "user biasa" bisa membaca object milik admin hanya dengan mengganti id, itu bukti **broken object-level authorization**. Jangan menganggap random UUID sebagai authorization control — UUID hanya membuat identifier lebih sulit ditebak.

Remediation.

Deny by default dan lakukan authorization check server-side pada setiap request terhadap object/action. Policy harus memakai trusted identity dari session/token yang tervalidasi, bukan role/object id dari client.

11. Burp Workflow per Vulnerability

Vulnerability	Baseline yang dicari	Variabel yang diubah	Proof minimum
SQLi	filter normal	one query parameter	result set/behavior berubah konsisten
XSS	plain reflection	one reflected value	benign JS executes
Command Injection	normal command result	one argument	additional command output
Traversal	allowed file	filename/path	harmless file outside base dir
Upload	allowed file upload	file type/name	unsafe serving behavior
Auth	valid student session	session identity	identity changes without legitimate auth
IDOR	own object	object id	other object returned without authorization

Evidence discipline.

Simpan request baseline dan modified request berdampingan. Catat hanya delta yang relevan. Finding yang baik harus bisa direproduksi tester lain tanpa tebakan.

```
python labs/finding-note.py module11-web-vulns/10-findings/idor.md \
--title "Broken object-level authorization" \
--category "Access Control / IDOR" \
--endpoint "GET /api/profile?id={id}" \
--evidence "Changing id=1 to id=2 returns another profile." \
--impact "A low-privilege user can read another user's profile." \
--remediation "Enforce object authorization server-side."
```

12. PortSwigger Web Security Academy — Guided Practice

Local lab bagus untuk memahami root cause dengan cepat. Setelah itu, gunakan Web Security Academy untuk variasi realistis. Mulai dari Apprentice labs — jangan lompat ke bypass kompleks sebelum bisa menjelaskan baseline dan root cause.

Topik	Learning path/rekomendasi awal
SQL injection	SQL injection → basic WHERE clause/login logic labs
XSS	Cross-site scripting → reflected XSS into HTML context

Command injection	OS command injection → simple case
Path traversal	Path traversal → simple case
File upload	File upload vulnerabilities → basic unrestricted upload lab
Authentication	Authentication vulnerabilities → logic/username-enumeration beginner labs
Access control	Access control → unprotected functionality/IDOR beginner labs

Rule latihan.

Gunakan credential, wordlist, dan payload yang disediakan oleh lab. Jangan memindahkan teknik rate-heavy seperti brute force ke website nyata.

13. Practical Challenge — Black-Box Mini Assessment

Jalankan local lab tanpa membaca app.py. Anggap source code tidak tersedia. Objective: temukan dan dokumentasikan minimal **empat** vulnerability berbeda dengan Burp.

- Create workspace dan start lab.
- Browse seluruh fungsi dan buat site map manual.
- Pilih endpoint satu per satu, kirim ke Repeater, dan ubah satu variabel per eksperimen.
- Temukan minimal 4 dari 7 vulnerability classes.
- Untuk setiap finding simpan baseline, modified request, response delta, impact, remediation.
- Stop lab dan cleanup container setelah selesai.

```
./labs/vuln-web-lab.sh cleanup
```

- [] Saya tidak membaca source sebelum challenge.
- [] Setiap finding memiliki baseline dan modified evidence.
- [] Saya dapat menjelaskan root cause tanpa menyebut payload saja.
- [] Impact saya berdasarkan apa yang terbukti, bukan asumsi liar.
- [] Remediation menyorot server-side root cause.
- [] Lab sudah di-cleanup.

14. Knowledge Check

- Kenapa blacklist karakter bukan defense utama SQL injection?
- Apa bedanya reflected, stored, dan DOM-based XSS?
- Kenapa HTML encoding tidak otomatis benar untuk JavaScript context?
- Mengapa shell=True berbahaya ketika input user digabungkan ke command string?
- Kenapa . . / filtering saja tidak cukup untuk path traversal?
- Kenapa Content-Type upload dari browser tidak boleh dipercaya?
- Apa bedanya authentication dan authorization?
- Kenapa UUID tidak memperbaiki IDOR dengan sendirinya?
- Apa arti "change one variable" dalam Repeater workflow?
- Apa bukti minimum yang cukup untuk command injection di lab?
- Kenapa impact finding tidak boleh melebihi evidence?
- Apa perbedaan root-cause remediation dan WAF mitigation?

15. Cheat Sheet

Tujuan	Local lab proof
SQLi	category=' OR 1=1--
Reflected XSS	<svg onload=alert(document.domain)>
Command injection	host=127.0.0.1;id
Path traversal	file=../../../../etc/hostname
Upload	harmless .html served same-origin
Auth/session	Cookie: user=student → user=admin
IDOR	id=1 → id=2

Jangan hafal payload.

Payload di atas hanya proof kecil untuk environment module ini. Skill yang transferable adalah mengenali trust boundary, context, server-side decision, dan evidence delta.

16. Exit Criteria

- [] Local lab dapat dibuat dan dibersihkan dengan aman.
- [] Bisa menemukan tujuh endpoint vulnerable melalui browser/Burp.
- [] Paham SQLi sebagai data-to-SQL-code boundary failure.
- [] Paham XSS berdasarkan output context, bukan sekadar <script>.
- [] Bisa menjelaskan command injection dan secure process invocation.
- [] Bisa menjelaskan canonical path validation.
- [] Bisa membuat checklist file-upload controls.
- [] Bisa membedakan AuthN, session integrity, AuthZ, dan IDOR.
- [] Sudah menyelesaikan minimal satu PortSwigger Apprentice lab untuk tiap kategori utama.
- [] Menghasilkan minimal 4 finding note dari black-box challenge.

17. Deliverables

```
11-web-vulnerabilities-i/
├── README.md
├── docx/Red-Team-Module-11-Web-Vulnerabilities-I-0marchy.docx
├── pdf/Red-Team-Module-11-Web-Vulnerabilities-I-0marchy.pdf
├── labs/
│   ├── workspace-init.sh    - scaffold workspace (00-scope ... 10-findings)
│   ├── finding-note.py     - generate finding note markdown
│   ├── vuln-web-lab.sh     - build/create/status/logs/cleanup container lab
│   └── vuln-web-lab/
│       ├── Dockerfile
│       └── app.py           - Flask app: 7 kelas vulnerability dalam satu service
```

18. References

- PortSwigger — SQL injection — <https://portswigger.net/web-security/sql-injection>
- PortSwigger — Cross-site scripting — <https://portswigger.net/web-security/cross-site-scripting>
- PortSwigger — OS command injection — <https://portswigger.net/web-security/os-command-injection>
- PortSwigger — Path traversal — <https://portswigger.net/web-security/file-path-traversal>
- PortSwigger — File upload vulnerabilities — <https://portswigger.net/web-security/file-upload>

- PortSwigger — Authentication vulnerabilities — <https://portswigger.net/web-security/authentication>
- PortSwigger — Access control/IDOR — <https://portswigger.net/web-security/access-control/idor>
- OWASP — SQL Injection Prevention Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- OWASP — Cross Site Scripting Prevention Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- OWASP — OS Command Injection Defense — https://cheatsheetseries.owasp.org/cheatsheets/OS_Command_Injection_Defense_Cheat_Sheet.html
- OWASP — File Upload Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html
- OWASP — Authentication Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- OWASP — Authorization Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

Next module:

12-web-vulnerabilities-ii — SSRF, XXE, SSTI, insecure deserialization, CORS, CSRF, JWT, OAuth, GraphQL, NoSQL injection, request smuggling, cache issues, race conditions, prototype pollution, dan business logic.